MINOR PROJECT

REPORT ON

CNN IMAGE CLASSIFICATION AND STREAMLIT APPLICATION

Submitted By

Project Title: CNN Image Classification and Streamlit Application

Student Name: Pranathi R

Selected Dataset: MNIST Handwritten Digit Dataset

Streamlit App Topic: Student Marks Calculator

Batch: Sep 2026

Date :01/11/2026

Chapter 2: Objective

The main objective of this project is to understand the working of Convolutional Neural Networks (CNNs) and develop an interactive web application using Streamlit. This project combines deep learning and Python application development to demonstrate two practical applications.

The project is divided into two parts:

2.1 CNN Image Classification

The objective of Part A is to develop and train a Convolutional Neural Network (CNN) model to recognize handwritten digits from images. The MNIST handwritten digit dataset is used for training and testing the model. It contains grayscale images of digits ranging from 0 to 9.

The CNN model learns important features and patterns from the input images through different layers, such as convolutional, pooling, and dense layers. After training, the model predicts the digit present in an image. Its performance is evaluated using test accuracy, test loss, and actual versus predicted labels.

Problem Addressed: Recognizing handwritten digits manually can be time-consuming. This project demonstrates how deep learning can automate handwritten digit recognition and classify images accurately.

2.2 Streamlit Application

The objective of Part B is to develop an interactive Student Marks Calculator using Streamlit. The application provides a simple interface where users can enter a student's name and marks in five subjects: Python, Machine Learning, DBMS, Mathematics, and Computer Networks.

Based on the entered marks, the application automatically calculates the total marks, percentage, and average. It also assigns a grade and displays the pass or fail status according to the subject-wise passing criteria. Input validation is included to ensure that the student name is not empty and marks are within the range of 0 to 100.

Problem Addressed: Calculating student marks, percentages, averages, and grades manually requires time and may result in calculation errors. This application automates the process and displays the results in a clear and user-friendly format.

2.3 Overall Objective

Through this project, the aim is to gain practical knowledge of Python programming, image preprocessing, deep learning, CNN architecture, model training, model evaluation, and Streamlit application development. It also demonstrates how machine learning and web application technologies can be used to solve different real-world problems.

Chapter 3: CNN Workflow

3.1 Dataset Description

For this project, the MNIST (Modified National Institute of Standards and Technology) dataset is used to perform handwritten digit classification. It is a commonly used dataset in image processing and deep learning. It contains 70,000 grayscale images of handwritten digits from 0 to 9. Out of these, 60,000 images are used for training and 10,000 images are used for testing. Each image has a dimension of 28 × 28 pixels and belongs to one of the 10 classes (0–9).

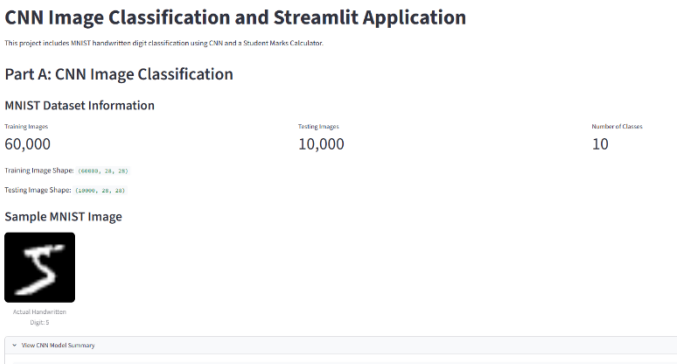


Figure 3.1: Sample handwritten digit images from the MNIST dataset.

3.2 Data Preprocessing

Before training the model, the images are pre-processed to make them suitable for CNN input. First, the MNIST dataset is loaded using TensorFlow Keras. The pixel values are converted into floating-point numbers and normalized by dividing them by 255, changing their range from 0–255 to 0–1. The images are then reshaped into the format (28, 28, 1), where 1 represents the grayscale channel. The labels are already represented as integer values from 0 to 9, so they are used directly during training.

3.3 CNN Model Architecture

A Convolutional Neural Network (CNN) is a deep learning model designed for image recognition and classification. It automatically extracts important features such as edges, shapes, and patterns from images.

The CNN model developed in this project uses two convolutional layers with 32 and 64 filters, respectively. Each convolutional layer is followed by a MaxPooling layer to reduce the feature-map dimensions. A Flatten layer converts the extracted features into a one-dimensional vector. A Dense layer with 64 neurons learns important patterns, while a Dropout layer with a rate of 0.5 helps reduce overfitting. Finally, an output Dense layer with 10 neurons and Softmax activation predicts the digit class.

3.4 Model Training

The model is compiled using the Adam optimizer, Sparse Categorical Crossentropy loss function, and accuracy metric. It is trained for 5 epochs with a batch size of 64. A validation split of 20% is used to monitor the model's performance during training. The model learns from the training images by adjusting its weights to improve classification accuracy.

3.5 Training and Validation Performance

Training and validation accuracy are recorded during each epoch to observe how well the model learns. The accuracy graph shows the classification performance during training and validation. Similarly, the loss graph represents the difference between actual labels and predicted outputs. These graphs help monitor the learning process and identify possible overfitting.

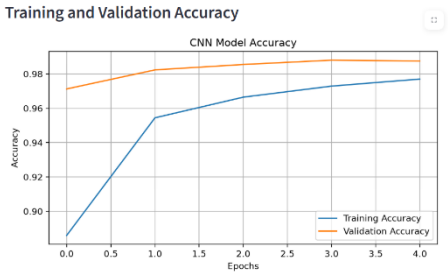


Figure 3.2: Training and validation accuracy.

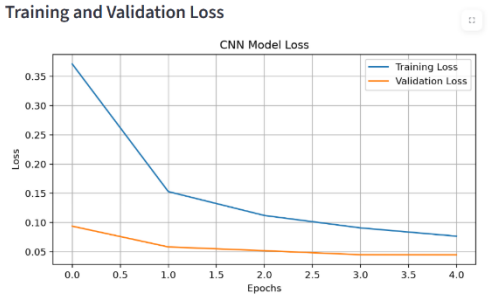


Figure 3.3: Training and validation loss.

3.6 Model Evaluation

After training, the model is evaluated using the 10,000 test images. It achieved a test accuracy of **98.81%** and a test loss of **0.0352**. Out of 10,000 test images, 9,881 were classified correctly and 119 were classified incorrectly. The actual and predicted labels are also compared to understand the model's classification performance.



Figure 3.4: Comparison of actual and predicted digit labels.

The results show that the CNN model successfully learned the features of handwritten digits and achieved high classification accuracy on the MNIST test dataset.

Chapter 4: Streamlit Application

4.1 Application Purpose

The second part of this project focuses on developing a Student Marks Calculator using Streamlit. Streamlit is a Python framework used to create interactive web applications without requiring extensive frontend development.

The purpose of this application is to simplify the process of calculating student academic results. It automatically calculates total marks, percentage, average, grade, and pass/fail status based on the marks entered by the user.

4.2 User Inputs

The application provides a simple and user-friendly interface where users can enter the following details:

1. Student Name
2. Python Marks
3. Machine Learning Marks
4. DBMS Marks
5. Mathematics Marks
6. Computer Networks Marks

Each subject mark must be between 0 and 100. After entering all the details, the user clicks the Submit button to calculate the result.

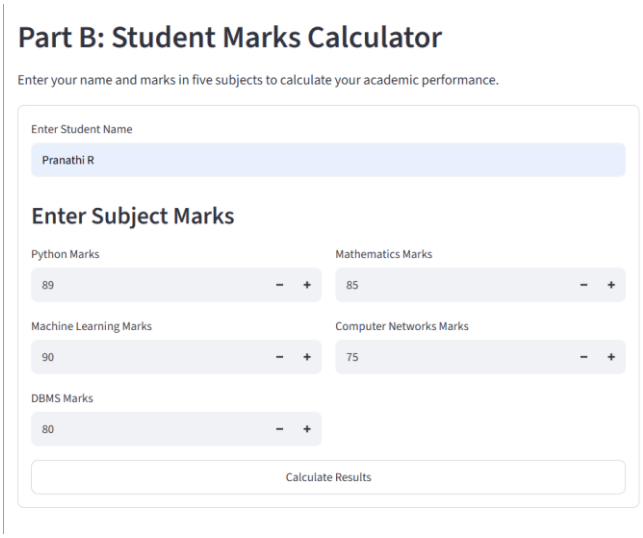


Figure 4.1: Student Marks Calculator input interface.

4.3 Processing and Calculations

After the user submits the details, the application performs the following operations:

- **Total Marks:** Calculates the sum of marks obtained in all five subjects.
- **Percentage:** Calculates the percentage based on the total marks out of 500.
- **Average:** Calculates the average marks across the five subjects.
- **Grade:** Assigns a grade based on the calculated percentage.
- **Pass/Fail Status:** Displays PASS if the student scores at least 35 marks in every subject; otherwise, it displays FAIL.

The application also validates the student name to ensure that it is not empty and restricts subject marks to the range of 0–100.

4.4 Application Output

After successful submission, the application displays the student's name, total marks, percentage, average, grade, and pass/fail status in a clear format.

For example, a student who obtains 419 marks out of 500 receives a percentage of 83.80%, an average of 83.80, a grade of A, and a PASS result.

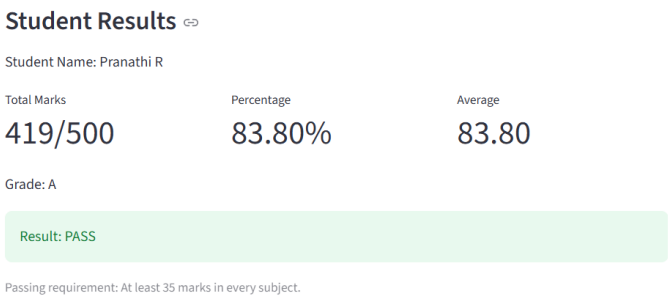


Figure 4.2: Calculated student result displayed by the application.

4.5 Features of the Application

- Simple and interactive web interface.
- Accepts student details and marks in five subjects.
- Validates student name and marks.
- Automatically calculates total, percentage, and average.Assigns grades based on percentage.
- Checks subject-wise passing criteria.Displays results in an organized format.

The Student Marks Calculator was successfully developed using Streamlit. It demonstrates how Python can be used to automate calculations and create interactive applications that are easy to use.

Chapter 5: Results And Observations

5.1 CNN Model Performance

The CNN model was successfully trained using the MNIST handwritten digit dataset for five epochs. It was evaluated using 10,000 test images and achieved a test accuracy of 98.81% with a test loss of 0.0352.

The model correctly classified 9,881 images, while 119 images were incorrectly classified. This shows that the model performs well in handwritten digit recognition.

Model Evaluation Results:

Metric	Result
Training Epochs	5
Test Images	10,000
Test Accuracy	98.81%
Test Loss	0.0352
Correct Predictions	9,881
Incorrect Predictions	119

5.2 Streamlit Application Output

The Student Marks Calculator was successfully developed using Streamlit. It accepts student details and marks in five subjects and automatically calculates the academic results.

Sample Application Result:Total Marks: 419/500 , Percentage: 83.80% , Average: 83.80 , Grade: A , Result: PASS

5.3 Observations

1. The CNN model successfully learned features from handwritten digit images.
2. The model achieved high accuracy on the test dataset. Accuracy and loss graphs helped monitor the training process.
3. Some images were incorrectly classified due to similarities in handwritten digits.
4. The Streamlit application performed calculations automatically and displayed results correctly.Input validation helped prevent invalid entries.

5.4 Important Findings

The project demonstrates that CNNs are effective for handwritten digit classification and can achieve high prediction accuracy. It also shows that Streamlit provides a simple way to develop interactive Python applications.

Chapter 6: Conclusion

6.1 Conclusion

This project successfully demonstrates the implementation of a Convolutional Neural Network (CNN) for handwritten digit classification and the development of an interactive web application using Streamlit.

In Part A, a CNN model was developed and trained using the MNIST handwritten digit dataset. The model achieved a test accuracy of 98.81% with a test loss of 0.0352. This demonstrates the ability of CNNs to identify and classify handwritten digits based on the features learned from images.

In Part B, a Student Marks Calculator was developed using Python and Streamlit. The application accepts student details and subject marks and automatically calculates total marks, percentage, average, grade, and pass/fail status. It provides a simple and user-friendly interface for calculating academic results.

Overall, both parts of the project were implemented successfully and provided practical knowledge of deep learning and web application development.

6.2 Learning Outcomes

Through this project, the following concepts and skills were learned:

1. Understood the MNIST dataset, image preprocessing, normalization, and reshaping techniques.
2. Gained practical knowledge of CNN architecture, its layers, model training, and evaluation.
3. Learned to analyze accuracy and loss graphs and compare actual and predicted image classifications.
4. Developed an interactive Streamlit application using Python, including input validation, mathematical calculations, and result display.

6.3 Challenges Faced and Future Improvements

1. Understanding CNN architecture and preparing image data through proper normalization and reshaping was challenging. This can be improved by exploring advanced CNN architectures and training techniques.
2. Monitoring training accuracy, validation accuracy, and loss graphs required understanding the model's learning process. Future improvements can focus on improving model performance and reducing incorrect predictions.
3. Developing an interactive Streamlit interface and implementing input validation, grade calculation, and pass/fail conditions required careful programming. The application can be enhanced with a more advanced user interface.
4. he project can be improved by allowing users to upload handwritten digit images, supporting additional datasets such as CIFAR-10, and integrating a database, performance charts, and PDF report downloads into the Student Marks Calculator.